Deep Learning - lecture 5

Recurrent Neural Networks

Prof. Stefano Carrazza

University of Milan and INFN Milan

Computational Graphs

Introducing computational graphs

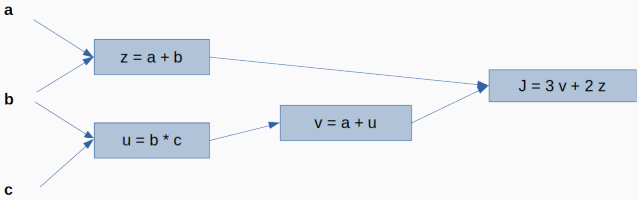
Let's suppose to compute the following expression:

$$J(a, b, c) = 3(a + bc) + 2(b + a)$$

Introducing computational graphs

Let's suppose to compute the following expression:

$$J(a, b, c) = 3(a + bc) + 2(b + a)$$

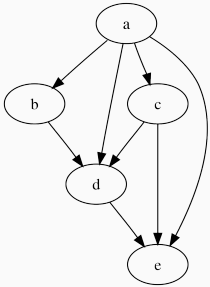


The evaluation of z and u can be performed in parallel.

Introducing computational graphs

From a technical point of view a computational graph:

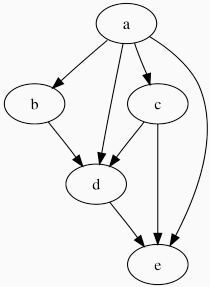
- formalizes the structure of a set of computations
- defines a chain of events
- it represents the basic rules for NNs



Introducing computational graphs

From a technical point of view a computational graph:

- formalizes the structure of a set of computations
- defines a chain of events
- it represents the basic rules for NNs



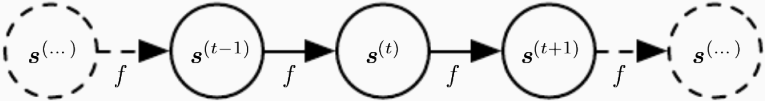
A graph is defined by a set of **vertices** (nodes) and **edges** (links).

A direct graph contains edge orientations and may contain **loops** (e.g. recurrency).

An **acyclic direct graph (DAG)** is a directed graph with no directed cycles.

Introducing computational graphs

For example, given a system represented by the graph:



we can determine the state $\boldsymbol{s}^{(t)}$ at any position t with:

$$\boldsymbol{s}^{(t)} = f(\boldsymbol{s}^{(t-1)}; \boldsymbol{w})$$

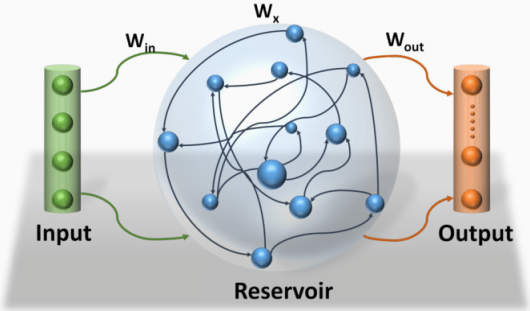
which can be **unfolded** (not involving recurrence):

$$\boldsymbol{s}^{(4)} = f(\boldsymbol{s}^{(3)}; \boldsymbol{w}) = f(f(f(\boldsymbol{s}^{(1)}; \boldsymbol{w}); \boldsymbol{w}); \boldsymbol{w})$$

Recurrent neural networks

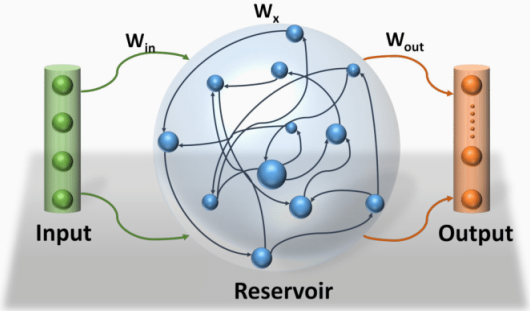
Recurrent neural networks

Let's consider an **Echo State Network**:



Recurrent neural networks

Let's consider an **Echo State Network**:



The behaviour is non-linear, the only weights that are modified during training are for the synapses that connect the hidden neurons to output neurons.

Recurrent neural networks

Suppose to operate on a **sequence** $\boldsymbol{x}^{(t)}$:

$$(\boldsymbol{x}^{(t)}, \boldsymbol{x}^{(t-1)}, \boldsymbol{x}^{(t-2)}, \ldots, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(1)})$$

How can we take advantage of the sequence correlation / causality?

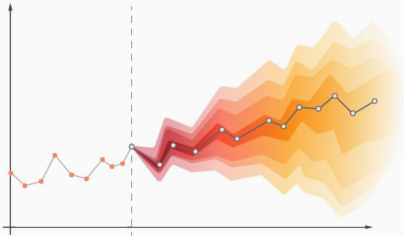
Recurrent neural networks

Suppose to operate on a **sequence** $\boldsymbol{x}^{(t)}$:

$$(\boldsymbol{x}^{(t)}, \boldsymbol{x}^{(t-1)}, \boldsymbol{x}^{(t-2)}, \ldots, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(1)})$$

How can we take advantage of the sequence correlation / causality?

Application examples:



- Time series predictions
- Speech/Handwriting recognition
- Text translation
- Anomaly detection
- Protein homology detection
- ...

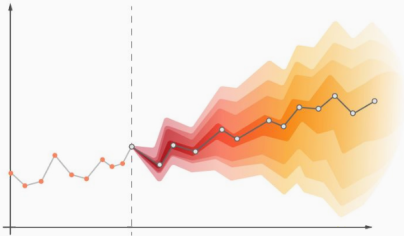
Recurrent neural networks

Suppose to operate on a **sequence** $\boldsymbol{x}^{(t)}$:

$$(\boldsymbol{x}^{(t)}, \boldsymbol{x}^{(t-1)}, \boldsymbol{x}^{(t-2)}, \ldots, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(1)})$$

How can we take advantage of the sequence correlation / causality?

Application examples:



- Time series predictions
- Speech/Handwriting recognition
- Text translation
- Anomaly detection
- Protein homology detection
- ...

$\Rightarrow$ Recurrent Neural Networks (RNNs), Rumelhart et al. 1986

Recurrent neural networks

Suppose to operate on a sequence of vectors $x^{(t)}$:

$$(\boldsymbol{x}^{(t)}, \boldsymbol{x}^{(t-1)}, \boldsymbol{x}^{(t-2)}, \ldots, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(1)})$$

A simple example of recurrent network with no outputs is defined by:

$$\boldsymbol{h}^{(t)} = f(\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}; \boldsymbol{w})$$

Recurrent neural networks

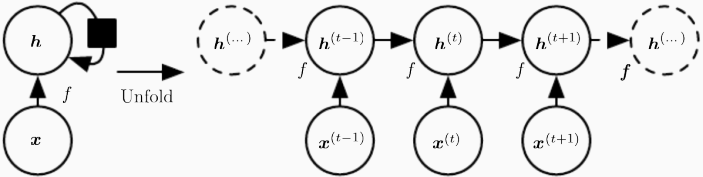
Suppose to operate on a sequence of vectors $x^{(t)}$:

$$(\boldsymbol{x}^{(t)}, \boldsymbol{x}^{(t-1)}, \boldsymbol{x}^{(t-2)}, \ldots, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(1)})$$

A simple example of recurrent network with no outputs is defined by:

$$\boldsymbol{h}^{(t)} = f(\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}; \boldsymbol{w})$$

Graphically:



The input $\boldsymbol{x}$ is incorporated into the state $\boldsymbol{h}$ which is passed forward through time.

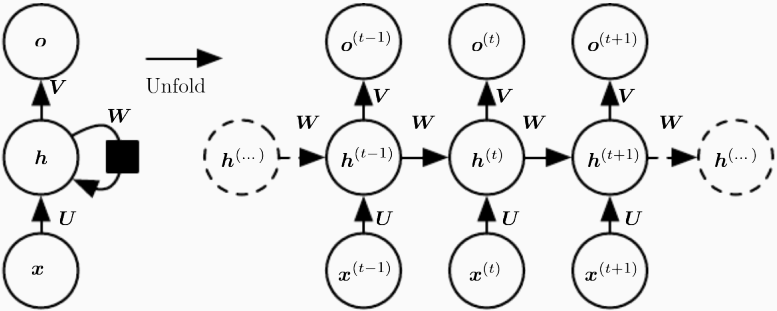
Recurrent neural networks

From a practical perspective, there are three RNN important design patterns:

Recurrent neural networks

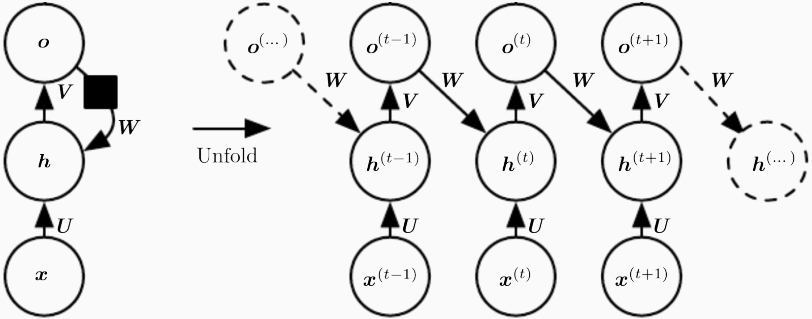
From a practical perspective, there are three RNN important design patterns:

1. RNN produces an output at each time step and have recurrent connections for hidden units.



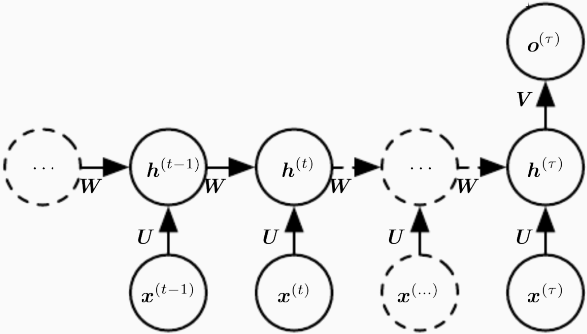
Recurrent neural networks

2. RNN produces an output at each time step and have recurrent connections only from the output at one time step to the hidden units at the next time step.



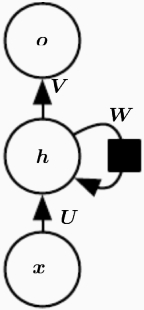
Recurrent neural networks

3. RNN with recurrent connections between hidden units, that read an entire sequence and then procedure a single output.



Example of RNN forward propagation

Let's consider design **1**, i.e. RNN produces an output at each time step and have recurrent connections for hidden units.



Forward propagation:

Assuming $\tanh$ activation function:

$$\begin{aligned} \boldsymbol{a}^{(t)} &= \boldsymbol{b} + \boldsymbol{W}\boldsymbol{h}^{(t-1)} + \boldsymbol{U}\boldsymbol{x}^{(t)} \\ \boldsymbol{h}^{(t)} &= \tanh(\boldsymbol{a}^{(t)}) \\ \boldsymbol{o}^{(t)} &= \boldsymbol{c} + \boldsymbol{V}\boldsymbol{h}^{(t)} \end{aligned}$$

where $\boldsymbol{b}$ and $\boldsymbol{c}$ re the bias vectors, and:

- $\boldsymbol{U}$ input-to-hidden connections
- $\boldsymbol{V}$ hidden-to-output connections
- $\boldsymbol{W}$ hidden-to-hidden connections

Example of RNN gradient evaluation

RNN gradients are evaluated using **back-propagation through time** (BPTT).

Considering a loss function where $\left\{\boldsymbol{y}^{(1)}, \ldots, \boldsymbol{y}^{(\tau)}\right\}$ are targets:

$$J\left(\left\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(\tau)}\right\}, \left\{\boldsymbol{y}^{(1)}, \ldots, \boldsymbol{y}^{(\tau)}\right\}\right) = \sum_{t} J^{(t)}$$

Example of RNN gradient evaluation

RNN gradients are evaluated using **back-propagation through time** (BPTT).

Considering a loss function where $\left\{\boldsymbol{y}^{(1)}, \ldots, \boldsymbol{y}^{(\tau)}\right\}$ are targets:

$$J\left(\left\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(\tau)}\right\}, \left\{\boldsymbol{y}^{(1)}, \ldots, \boldsymbol{y}^{(\tau)}\right\}\right) = \sum_t J^{(t)}$$

As usual, starting from the last layer:

$$\left(\nabla_{\boldsymbol{o}^{(t)}} J\right)_i = \frac{\partial J}{\partial o_i^{(t)}} = \frac{\partial J}{\partial J^{(t)}} \frac{\partial J^{(t)}}{\partial o_i^{(t)}} = \frac{\partial J^{(t)}}{\partial o_i^{(t)}}, \qquad \frac{\partial J}{\partial J^{(t)}} = 1$$

Example of RNN gradient evaluation

RNN gradients are evaluated using **back-propagation through time** (BPTT).

Considering a loss function where $\left\{\boldsymbol{y}^{(1)}, \ldots, \boldsymbol{y}^{(\tau)}\right\}$ are targets:

$$J\left(\left\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(\tau)}\right\}, \left\{\boldsymbol{y}^{(1)}, \ldots, \boldsymbol{y}^{(\tau)}\right\}\right) = \sum_t J^{(t)}$$

As usual, starting from the last layer:

$$\left(\nabla_{\boldsymbol{o}^{(t)}} J\right)_i = \frac{\partial J}{\partial o_i^{(t)}} = \frac{\partial J}{\partial J^{(t)}} \frac{\partial J^{(t)}}{\partial o_i^{(t)}} = \frac{\partial J^{(t)}}{\partial o_i^{(t)}}, \qquad \frac{\partial J}{\partial J^{(t)}} = 1$$

and using design **1** where $\boldsymbol{o}^{(\tau)} = \boldsymbol{c} + \boldsymbol{V}\boldsymbol{h}^{(\tau)}$:

$$\nabla_{\boldsymbol{h}^{(\tau)}} J = \frac{\partial J}{\partial \boldsymbol{h}^{(\tau)}} = \frac{\partial \boldsymbol{o}^{(\tau)}}{\partial \boldsymbol{h}^{(\tau)}} \frac{\partial J^{(\tau)}}{\partial \boldsymbol{o}^{(\tau)}} = \boldsymbol{V}^T \nabla_{\boldsymbol{o}^{(\tau)}} J$$

Example of RNN gradient evaluation

Thus:

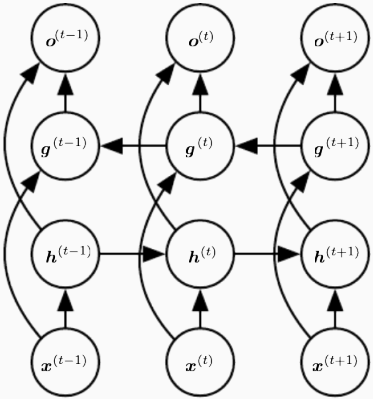
$$\nabla_{\boldsymbol{h}^{(t)}} J = \left(\frac{\partial \boldsymbol{h}^{(t+1)}}{\partial \boldsymbol{h}^{(t)}}\right)^T (\nabla_{\boldsymbol{h}^{(t+1)}} J) + \left(\frac{\partial \boldsymbol{o}^{(t)}}{\partial \boldsymbol{h}^{(t)}}\right)^T (\nabla_{\boldsymbol{o}^{(t)}} J)$$

Using design **1**, where $\boldsymbol{o}^{(t)} = \boldsymbol{c} + \boldsymbol{V}\boldsymbol{h}^{(t)}$ and $\boldsymbol{h}^{(t)} = \tanh(\boldsymbol{a}^{(t)})$:

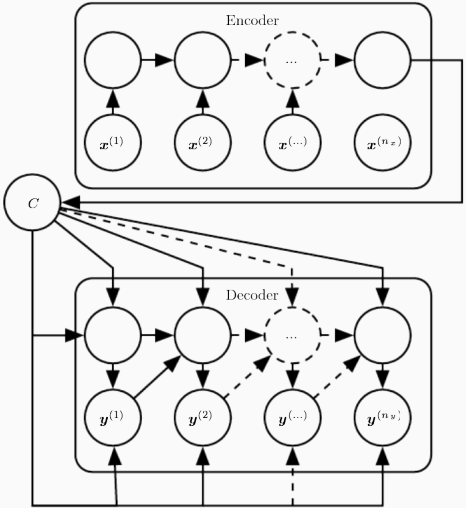
$$\nabla_{\boldsymbol{h}^{(t)}} J = \boldsymbol{W}^T \operatorname{diag}\left(1 - \left(\boldsymbol{h}^{(t+1)}\right)^2\right) (\nabla_{\boldsymbol{h}^{(t+1)}} J) + \boldsymbol{V}^T (\nabla_{\boldsymbol{o}^{(t)}} J)$$

Bidirectional RNNs

Information is propagated forward and **backward** in time.

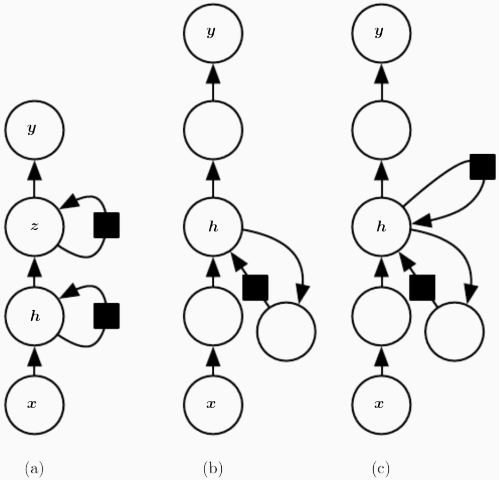


Encoder-decoder sequence-to-sequence architectures



Deep RNNs

Deep Recurrent Networks



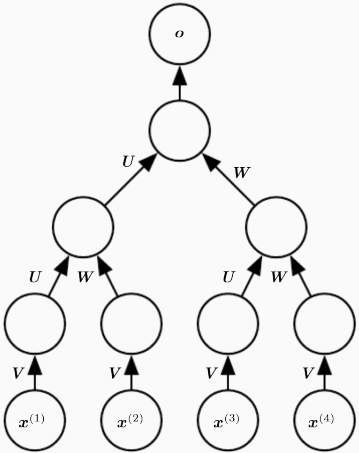
How to make a RNN deep?

- (a) multiple hidden layers
- (b) deeper computations
- (c) skip connections

Recursive Networks

RvNN is another generalization of deep recurrent networks with:

- Structured predictions.
- Reduced depth.
- Potentially simpler to train.
- Successful applications in NLP. (e.g. word embedding)



DRNNs issues

Deep RNN is designed to propagate "long-term" dependencies, $\boldsymbol{h}^{(t)}$.

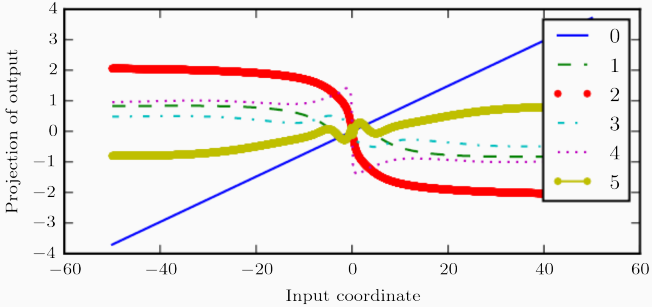
However, recurrent evaluations over many hidden layers deteriorate the gradient.

RNN gradients tend to vanish or diverge, for example:

$$\begin{aligned}
\nabla_{\boldsymbol{W}} J &= \sum_t \sum_i \left(\frac{\partial J}{\partial h_i^{(t)}} \right) \nabla_{\boldsymbol{W}^{(t)}} h_i^{(t)} \\
&= \sum_t \operatorname{diag} \left(1 - \left(\boldsymbol{h}^{(t)} \right)^2 \right) (\nabla_{\boldsymbol{h}^{(t)}} J) \boldsymbol{h}^{(t-1)^T} \\
&\propto \prod_i \frac{\partial \sigma_h}{\partial h_i} \to 0 \qquad \text{(with non-linear activation functions)}
\end{aligned}$$

DRNNs issues

Example: repeated nonlinear function composition.



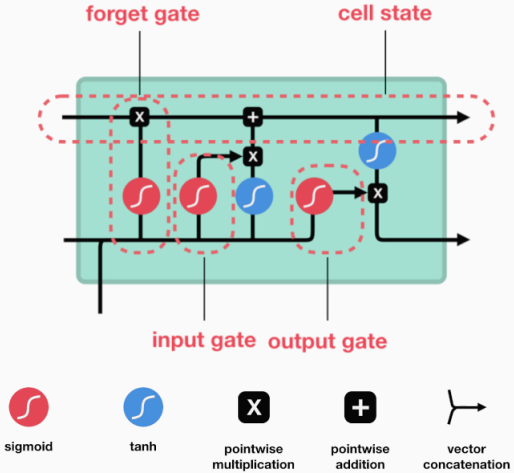
RNN layers that get a small gradient update stop learning (usually earlier layers).

- RNNs **forget** information from long sequences.
- RNNs have a **short-term memory** issue.

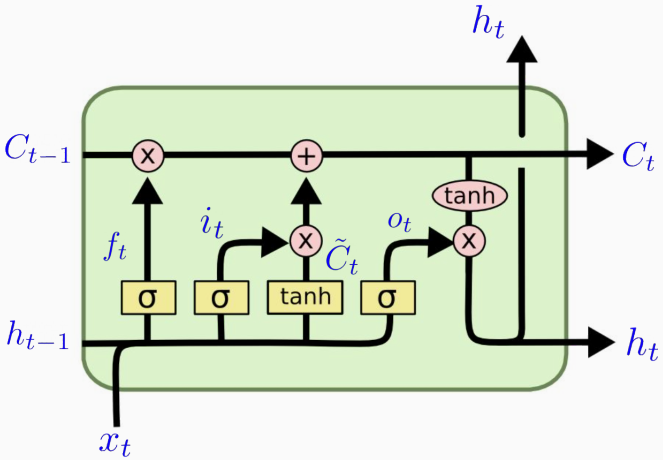
Fixing short-term memory

Long short-term memory

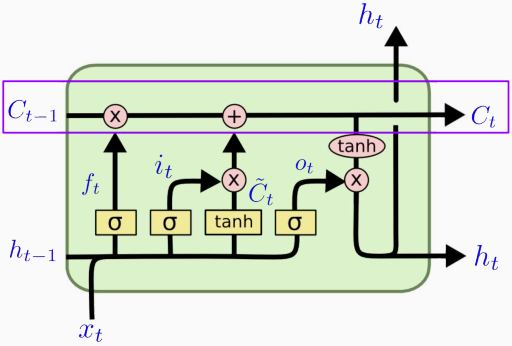
Long short-term memory (LSTM) networks introduce the concept of **gates** that regulate the flow of information.



LSTM



LSTM cell state

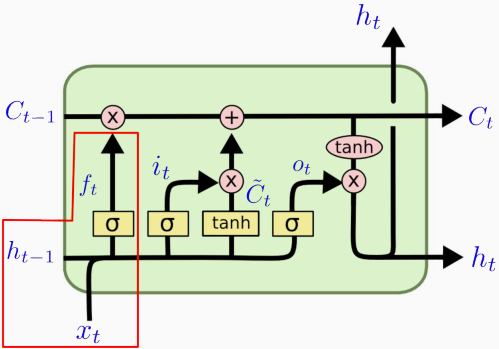


The cell state stores the relevant information over time:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

The **forget** and the **input** gates decide what we forget and remember.

LSTM forget gate

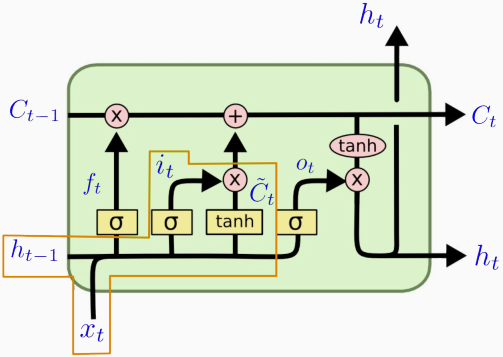


The **forget gate** decides what **information** we are going to **forget** and **keep** from **cell state**.

$$f_t = \sigma_g(W_f x_t + U_f h_{t-1} + b_f)$$

Takes the input x_t, hidden state h_{t-1} and $\sigma_g \in [0, 1]$ (sigmoid).

LSTM input state

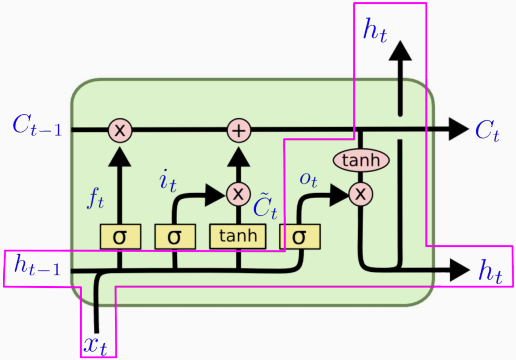


The **input gate** chooses the **new information** to store in the **cell state**.

$$
\begin{aligned}
i_t &= \sigma_g(W_i x_t + U_i h_{t-1} + b_i) \\
\tilde{C}_t &= \sigma_c(W_c x_t + U_c h_{t-1} + b_c)
\end{aligned}
$$

The **sigmoid** (σ_g) decides which values to update, the **tanh** (σ_h) builds the new candidate $\tilde{C}_t$.

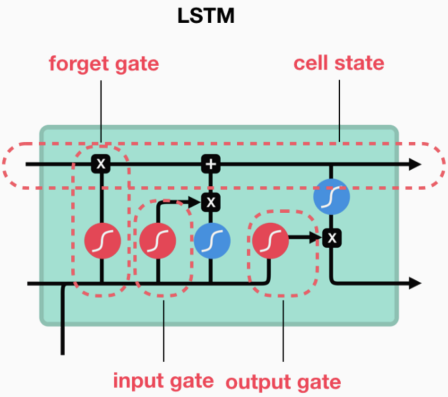
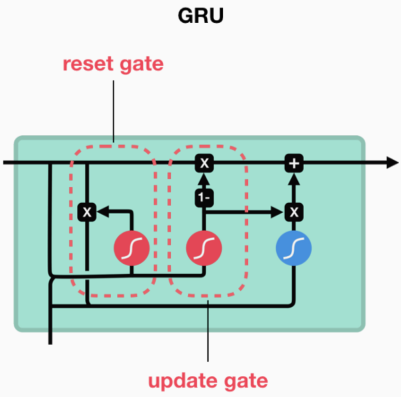
LSTM output state



The **output gate** post-processes the **cell state**.

$$
\begin{aligned}
o_t &= \sigma_g(W_o x_t + U_o h_{t-1} + b_o) \\
h_t &= o_t \sigma_h(C_t)
\end{aligned}
$$

The **sigmoid** (σ_g) chooses the information from the cell state, the **tanh** (σ_h) normalizes it.

Gated recurrent unit

: similar to LSTMs with a reset and update gates thus a smaller number of parameters.